Report of Malloc Library Project

Name: Ziyu Shi NetID: zs148

In this report, I will explain my implementation of my two versions of malloc() and free() with C, and will conduct a performance experimentation and analysis on my two versions of malloc().

1. An overview of my implementation

In this section, I will explain the data structure and functions I implemented in this project in detail. I created a structure named "Metadata" to store the information I need to manage the blocks in the heap, whether they are allocated or not. "Metadata" will store the size of the block (excluding the size of Metadata itself), the available status to check if this block is available for allocation, and its next and the previous blocks (if exist). The detailed structure is exhibited below.

```
typedef struct metadata_t {
  size_t size;
  int available; // 1 is free, 0 is allocated
  struct metadata_t* next; // next free block
  struct metadata_t* prev; // previous free block
} Metadata;
```

In this project, I am tasked to implement two versions of malloc() and free(), which are First Fit and Best Fit.

To achieve these two allocation policies, I implement a **free block list** to maintain the position of blocks and to improve the performance when looking for the suitable blocks instead of storing all blocks.

I implement **add_free_block** and **remove_free_block** functions to add one blocks when this block has been freed and to remove one block from the free block list when this block has been allocated.

Based on the consideration of the situation that there is no blocks existing in the heap at the very beginning of the program, I implement a **allocate_block** function to call **sbrk()** to ask for more space in the heap, and return its **Metadata** as its management unit to malloc functions (ff_malloc or bf_malloc).

To improve the space utilization, I implement a **split_block** to reuse the block if the size of this block is larger than the sum of the size requested by the user and the size of Metadata. **I will split a part of space from the back of this block and allocate this part of space to the user**, which ensures the correctness of relative

position of the original block because I do not have to change the next and previous pointer it owns.

To improve the performance of searching, I implement a **merge_blocks** function to merge two blocks if these two blocks is near to each other. I will change the size of the block at smaller position to the sum of these two blocks and remove the block at larger position from the free block list.

To achieve the two versions of malloc, I implement two function to find the corresponding block. First Fit is realized by **find_ff** function, which is shown below.

```
void * find_ff(size_t size) {
  Metadata * cur_block = first_free_block;
  while(cur_block != NULL && cur_block->size < size) {
    cur_block = cur_block->next;
  }
  if(cur_block == NULL) {
    return NULL;
  }
  return cur_block;
}
```

Best Fit is realized by **find_bf** function, which is shown below.

```
void * find_bf(size_t size) {
  Metadata * cur_block = first_free_block;
  Metadata * bf_block = NULL;
  while(cur_block != NULL) {
    if(cur_block->size >= size) {
      if(bf_block == NULL || bf_block->size > cur_block->size) {
        bf_block = cur_block;
        if(bf_block->size == size) {
          break;
        }
      }
    }
    cur_block = cur_block->next;
  }
  return bf_block;
}
```

2. Results and analysis

2.1 Results

In this section, I will exhibit the results of the performance experimentation and the corresponding explanation. The metrics of interest in this experimentation is the

run-time of program in different patterns of allocation policies and the fragmentation ratio of the free space in the heap and the entire data segment space. The result is shown in the table below. The data in the table is the average of the results of three executions of the program.

Patterns	First Fit		Best Fit	
	Execution Time/s	Fragmentation	Execution Time/s	Fragmentation
Equal	**15.60**	**0.450**	**15.61**	**0.450**
Small	**5.96**	**0.066**	**1.36**	**0.021**
Large	**65.14**	**0.122**	**55.63**	**0.045**

2.2 Analysis

The pattern of equal_size_allocs allocates the same number of bytes (128) in every call of malloc. Therefore, the execution time and the fragmentation ratio in this pattern keeps almost the same in two allocation policies.

The pattern of small_range_rand_allocs will call malloc to allocate random size which ranges from 128 to 512 bytes. In this pattern, the execution time in Best Fit is significantly better than that in First Fit. **From my perspective, in this pattern, because of the small size of data, First Fit need to frequently call allocate_block function to allocate new space from the heap, but Best Fit has no need to do this.** For example, First Fit has 3 blocks. These blocks respectively have 160, 128, 256 bytes of size. If the order of request for allocation size is 80, 160, 256, then the program has to call allocate_block to allocate new space. But for Best Fit, these is no need to call allocate_block. Because of the small data size, the overhead caused by allocation of new space takes the dominant position in this pattern. Due to the policy Best Fit follows, the space utilization in Best Fit is better than that in First Fit. Best Fit ensures that every allocation selects the most suitable block as the allocated block.

The pattern of large_range_rand_allocs will call malloc to allocate random size which ranges from 32 to 64K bytes. The result shows that the fragmentation in Best Fit is better that in First Fit because Best Fit will find the most suitable block to allocate. About the execution time, I assume that maybe the merge_blocks in First Fit is frequently called because the relatively unsuitable block size freed by ff_free(), and this process incurs a corresponding overhead, which is more

dominant during the execution than the searching policy of Best Fit. Therefore, Best Fit is faster than First Fit in this situation.

In my opinion, Best Fit is suitable for small size of data. Because in this situation the frequent calls of malloc incurs more serious overhead than searching for the best block. First Fit is suitable for large size of data, because if the data size reaches a certain level, searching for the best block will take the dominant position in the execution of program. As a result, I will recommend First Fit because the data size in our daily life is typically large size where First Fit dominates.